Experiment No. 04

Comparative Study of Regularization Techniques using Logistic Regression and Neural Networks

Student Lab Worksheet

Course	Advanced Neural Network Architectures (2306411T)
Duration	2 Hours
CO Mapping	CO1 – Explain basics of neural computation & optimization CO4 – Design end-to-end deep learning models
RBT Level	L3 (Apply)
Student Name	Gangotri Datta Kompalwar
Roll No.	202402070020
Batch / Date	C3/ 04-08-2026

Software Required

- Python 3.11
- Google Colab / VS Code
- TensorFlow / Keras
- Scikit-Learn
- NumPy, Pandas
- Matplotlib, Seaborn

1. Aim

To implement and compare the effect of regularization techniques — L1, L2, Dropout, Batch Normalization, and Early Stopping — on a Logistic Regression model and a Neural Network, using the Breast Cancer Wisconsin dataset, and to study how each technique improves generalization and reduces overfitting.

2. Learning Objectives

By the end of this lab, you will be able to:

- Explain the difference between overfitting and underfitting
- Apply L1 and L2 regularization to a Logistic Regression model
- Build a feed-forward Neural Network in Keras/TensorFlow
- Apply Dropout, Batch Normalization, and Early Stopping to a Neural Network
- Evaluate models using Accuracy, Precision, Recall, F1-score, Confusion Matrix, and ROC-AUC
- Compare 8 models and conclude which technique generalizes best

3. Dataset

Breast Cancer Wisconsin Dataset (built into scikit-learn — no download required).

Samples	Features	Classes
569	30 (numeric)	2 — 0: Malignant, 1: Benign

4. Theory (Read Before You Start)

4.1 Overfitting vs Underfitting

Overfitting: the model memorizes training data (e.g. Train Acc 99%, Test Acc 72%) — poor generalization.

Underfitting: the model fails to learn patterns at all (e.g. Train Acc 65%, Test Acc 62%).

Ideal model: Train and Test accuracy are both high and close to each other (e.g. 95% / 94%).

4.2 L1 Regularization (Lasso)

Adds the sum of absolute weight values to the loss function: Loss = Original Loss + $\lambda \times \Sigma|w|$

- Produces sparse models — some weights become exactly zero
- Effectively performs feature selection
- Useful in medical diagnosis, gene selection, text classification

4.3 L2 Regularization (Ridge)

Adds the sum of squared weight values to the loss function: Loss = Original Loss + $\lambda \times \Sigma w^2$

- Shrinks weights but rarely makes them exactly zero
- Produces smoother learning and is the most commonly used regularizer

4.4 Dropout

Randomly deactivates a fraction of neurons during each training step (e.g. with Dropout = 0.5 on a layer of 100 neurons, only 50 remain active for that step).

- Prevents neurons from co-adapting
- Improves generalization to unseen data

4.5 Batch Normalization

Normalizes the activations of a layer within each mini-batch.

- Speeds up training
- Stabilizes gradients
- Allows use of a higher learning rate
- Often improves final accuracy

4.6 Early Stopping

Monitors validation loss during training and stops automatically once it stops improving, restoring the best weights seen so far.

- Prevents overfitting from training too many epochs
- Reduces unnecessary training time

5. Problem Statement

Develop a binary classifier for breast cancer diagnosis using (1) Logistic Regression and (2) a Deep Neural Network. Study the effect of L1, L2, Dropout, Batch Normalization, and Early Stopping on model performance, and identify which technique gives the best generalization.

6. Experimental Design (Flow)

Load Dataset → EDA → Train/Test Split (80/20) → Feature Scaling → Model Building → Training → Evaluation → Comparison

7. Procedure — Step-by-Step Tasks

Work through each task in a single Colab / Jupyter notebook. Run each cell, record the requested outputs, and fill in the tables in Section 8 as you go.

Task 1: Import Libraries and Load the Dataset

```
import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import roc_auc_score
from sklearn.metrics import roc_curve

import tensorflow as tf

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.regularizers import l2
```

Task 2: Exploratory Data Analysis (EDA)

```
dataset = load_breast_cancer()

df = pd.DataFrame(dataset.data, columns=dataset.feature_names)
df["Target"] = dataset.target

df.head()


cancer = load_breast_cancer()

print("Target Classes:")
print(cancer.target_names)
print("Dataset Shape:", df.shape)

print("\nMissing Values in Dataset")
display(df.isnull().sum())

print("\nTarget Class Distribution")
display(df["Target"].value_counts())

plt.figure(figsize=(5,4))
sns.countplot(x="Target", data=df)

plt.title("Target Class Distribution")
plt.xlabel("Target")
plt.ylabel("Count")

plt.show()
```

Task 3: Train-Test Split and Feature Scaling

```
X = df.drop("Target", axis=1)
y = df["Target"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
```

```
    test_size=0.2,
    random_state=42
)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print("Training Data Shape :", X_train.shape)
print("Testing Data Shape :", X_test.shape)
```

Task 4: Logistic Regression — Without Regularization

```
result = []

def evaluate_model(model_name, model, X_train, X_test, y_train, y_test):

    train_prediction = model.predict(X_train)
    test_prediction = model.predict(X_test)

    train_accuracy = accuracy_score(y_train, train_prediction)
    test_accuracy = accuracy_score(y_test, test_prediction)

    precision = precision_score(y_test, test_prediction)
    recall = recall_score(y_test, test_prediction)
    f1 = f1_score(y_test, test_prediction)

    overfit_gap = train_accuracy - test_accuracy

    cm = confusion_matrix(y_test, test_prediction)

    print(f"\n{model_name}")
    print("-" * 40)
```

```
    print("Train Accuracy :", train_accuracy)
    print("Test Accuracy  :", test_accuracy)
    print("Precision      :", precision)
    print("Recall         :", recall)
    print("F1 Score       :", f1)

    print("\nConfusion Matrix")
    print(cm)

    result.append([
        model_name,
        train_accuracy,
        test_accuracy,
        precision,
        recall,
        f1,
        overfit_gap
    ])
```

Task 5: Logistic Regression — with L1 Regularization

```
lr_model = LogisticRegression(
    penalty=None,
    max_iter=5000,
    random_state=42
)

lr_model.fit(X_train, y_train)

evaluate_model(
    "Logistic Regression",
    lr_model,
    X_train,
    X_test,
```

```
    y_train,
    y_test
)
lr_l1_model = LogisticRegression(
    penalty="l1",
    solver="liblinear",
    C=1.0,
    max_iter=5000,
    random_state=42
)

lr_l1_model.fit(X_train, y_train)

evaluate_model(
    "Logistic Regression (L1)",
    lr_l1_model,
    X_train,
    X_test,
    y_train,
    y_test
)
```

Task 6: Logistic Regression — with L2 Regularization

```
lr_l2_model = LogisticRegression(
    penalty="l2",
    solver="liblinear",
    C=1.0,
    max_iter=5000,
    random_state=42
)

lr_l2_model.fit(X_train, y_train)
```

```
evaluate_model(
    "Logistic Regression (L2)",
    lr_l2_model,
    X_train,
    X_test,
    y_train,
    y_test
)
```

Task 7: Build a Neural Network — Without Regularization (Architecture: 30→64→32→1)

```
def build_model(use_dropout=False, use_batchnorm=False, use_l2=False):

    regularizer = l2(0.001) if use_l2 else None

    model = Sequential()

    model.add(Dense(
        64,
        activation="relu",
        kernel_regularizer=regularizer,
        input_shape=(30,)
    ))

    if use_batchnorm:
        model.add(BatchNormalization())

    if use_dropout:
        model.add(Dropout(0.5))

    model.add(Dense(
        32,
        activation="relu",
        kernel_regularizer=regularizer
```

```
    ))

    if use_batchnorm:
        model.add(BatchNormalization())

    if use_dropout:
        model.add(Dropout(0.5))

    model.add(Dense(1, activation="sigmoid"))

    model.compile(
        optimizer="adam",
        loss="binary_crossentropy",
        metrics=["accuracy"]
    )

    return model
def evaluate_neural_network(model_name, model):

    train_prediction = (model.predict(X_train) > 0.5).astype(int)
    test_prediction = (model.predict(X_test) > 0.5).astype(int)

    train_accuracy = accuracy_score(y_train, train_prediction)
    test_accuracy = accuracy_score(y_test, test_prediction)

    precision = precision_score(y_test, test_prediction)
    recall = recall_score(y_test, test_prediction)
    f1 = f1_score(y_test, test_prediction)

    overfitting_gap = train_accuracy - test_accuracy

    cm = confusion_matrix(y_test, test_prediction)
```

```
    print(f"\n{model_name}")
    print("-" * 40)
    print("Train Accuracy :", train_accuracy)
    print("Test Accuracy  :", test_accuracy)
    print("Precision      :", precision)
    print("Recall         :", recall)
    print("F1 Score       :", f1)

    print("\nConfusion Matrix")
    print(cm)

    result.append([
        model_name,
        train_accuracy,
        test_accuracy,
        precision,
        recall,
        f1,
        overfitting_gap
    ])
nn_model = build_model()

history = nn_model.fit(
    X_train,
    y_train,
    epochs=50,
    batch_size=32,
    validation_split=0.2,
    verbose=0
)

evaluate_neural_network(
    "Neural Network",
```

```
    nn_model
)
```

Task 8: Neural Network — with Dropout

```
dropout_model = build_model(use_dropout=True)

history_dropout = dropout_model.fit(
    X_train,
    y_train,
    epochs=50,
    batch_size=32,
    validation_split=0.2,
    verbose=0
)

evaluate_neural_network(
    "Neural Network + Dropout",
    dropout_model
)
```

Task 9: Neural Network — with Batch Normalization

```
batchnorm_model = build_model(use_batchnorm=True)

history_batchnorm = batchnorm_model.fit(
    X_train,
    y_train,
    epochs=50,
    batch_size=32,
    validation_split=0.2,
    verbose=0
)
```

```
evaluate_neural_network(
    "Neural Network + Batch Normalization",
    batchnorm_model
)
```

Task 10: Neural Network — with Early Stoppin

```
early_stopping = EarlyStopping(
    monitor="val_loss",
    patience=5,
    restore_best_weights=True
)

early_model = build_model()

history_early = early_model.fit(
    X_train,
    y_train,
    epochs=100,
    batch_size=32,
    validation_split=0.2,
    callbacks=[early_stopping],
    verbose=0
)

evaluate_neural_network(
    "Neural Network + Early Stopping",
    early_model
)
```

Task 11: Neural Network — All Techniques Combined (Dropout + BatchNorm + L2 + EarlyStopping)

```
combined_model = build_model(
    use_dropout=True,
```

```
    use_batchnorm=True,
    use_l2=True
)

history_combined = combined_model.fit(
    X_train,
    y_train,
    epochs=100,
    batch_size=32,
    validation_split=0.2,
    callbacks=[early_stopping],
    verbose=0
)

evaluate_neural_network(
    "Neural Network + Combined Techniques",
    combined_model
)
```

Task 12: **Compare All Models**

```
comparison_df = pd.DataFrame(
    result,
    columns=[
        "Model",
        "Train Accuracy",
        "Test Accuracy",
        "Precision",
        "Recall",
        "F1 Score",
        "Overfitting Gap"
    ]
)
```

```
print("Model Performance Comparison")
display(comparison_df)

comparison_df = comparison_df.sort_values(
    by="Test Accuracy",
    ascending=False
).reset_index(drop=True)

print("\nModels Ranked by Test Accuracy")
display(comparison_df)
plt.figure(figsize=(10,5))

plt.bar(
    comparison_df["Model"],
    comparison_df["Test Accuracy"]
)

plt.title("Test Accuracy of Different Models")
plt.xlabel("Models")
plt.ylabel("Test Accuracy")

plt.xticks(rotation=30)

plt.grid(axis="y", linestyle="--", alpha=0.5)

plt.show()
```

8. Results

8.1 Model Performance Comparison

Model	Train Acc	Test Acc	Overfitting Gap
Logistic Regression	1.0000	0.9386	0.0614
Logistic Regression (L1)	0.9890	0.9737	0.0153
Logistic Regression (L2)	0.9868	0.9737	0.0131
Neural Network	0.9934	0.9825	0.0110
Neural Network + Dropout	0.9868	0.9912	-0.0044
Neural Network + Batch Normalization	0.9912	0.9649	0.0263
Neural Network + Early Stopping	0.9780	0.9561	0.0219
Neural Network + Combined Techniques	0.8484	0.8509	-0.0025

8.2 Observation Table

Experiment	Accuracy	Precision	Recall	F1
Neural Network + Dropout	0.991228	0.986111	1.000000	0.993007
Neural Network	0.982456	0.972603	1.000000	0.986111

Experiment	Accuracy	Precision	Recall	F1
Logistic Regression (L2)	0.973684	0.972222	0.985915	0.979021
Logistic Regression (L1)	0.973684	0.985714	0.971831	0.978723
Neural Network + Batch Normalization	0.964912	0.958904	0.985915	0.972222
Neural Network + Early Stopping	0.956140	0.971429	0.957746	0.964539
Logistic Regression	0.938596	0.984848	0.915493	0.948905
Neural Network + Combined Techniques	0.850877	0.813953	0.985915	0.891720

9. Result / Conclusion

In this experiment, Logistic Regression and Neural Network models were trained using different regularization techniques on the Breast Cancer dataset. The results showed tha**t** Dropout gave the highest test accuracy and the smallest overfitting gap among all the techniques. The combined regularization model did not perform better than the individual Dropout model, showing that using multiple techniques together does not always improve performance. The Neural Network achieved better accuracy than Logistic Regression because it was able to learn the data patterns more effectively. Overall, regularization helped reduce overfitting and improved the performance of the models, with Dropout being the most effective technique in this experiment.

10. Viva Questions

Be ready to answer these verbally at the end of the lab:

1. Define overfitting and underfitting with examples.

2. Why is regularization needed in machine learning models?

3. What is the mathematical difference between L1 and L2 regularization?

4. Why does L1 regularization produce sparse (zero) weights while L2 does not?

5. Why does Dropout work as a regularizer? What does 'co-adaptation' mean?

6. What problem does Batch Normalization solve, and how?

7. How does Early Stopping decide when to stop training?

8. Why is feature scaling (StandardScaler) necessary before training these models?

9. Why use Logistic Regression at all when Neural Networks are more powerful?

10. Based on your results, which regularization technique performed best on this dataset, and why do you think that is?

11. Assignment (To Be Submitted Separately)

Repeat this entire experiment on the Heart Disease dataset (UCI Machine Learning Repository or Kaggle). Compare Logistic Regression and a Neural Network with L1, L2, Dropout, Batch Normalization, and Early Stopping, exactly as done above.

Submit a comparative analysis report containing:

- All graphs generated (loss curves, accuracy comparison, ROC curves)
- All evaluation metrics for every model
- A written conclusion on which technique generalized best on this new dataset